

InverseProperty Attribute in Entity Framework Core

In this article, I will discuss the **InverseProperty Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Index Attributes in Entity Framework Core](#) with Examples.

What is InverseProperty Attribute in Entity Framework Core?

The InverseProperty attribute in Entity Framework Core explicitly defines relationships between navigation properties when the relationships might be ambiguous or when EF Core's default conventions cannot determine them correctly. This attribute is especially useful when multiple navigation properties exist between the same two entities, and EF Core needs help determining which properties correspond to each other.

Example to Understand the InverseProperty Attribute in EF Core

Let us understand the **InverseProperty Attribute in EF Core** with one example. To understand this concept, we will create two Entities: **Course** and **Teacher**. Here, a teacher can teach multiple courses, but a course has one online and one offline teacher, making it a scenario where multiple relationships exist between the two entities.

In our example, the **Teacher Entity will be the Principal Entity**, and the **Course Entity will be the Dependent Entity**. We will create the foreign keys inside the Dependent Entity to establish the relationships.

Teacher Entity

Create a class file named **Teacher.cs** within the Entities folder and then copy and paste the following code. In this Teacher class, we have an OnlineCourses collection navigation property, which establishes a one-to-many relationship between Teacher and Course, i.e., one teacher can teach multiple online courses.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; }
        public string? Name { get; set; }
        public ICollection<Course>? OnlineCourses { get; set; }
    }
}
```

Course Entity:

Create another class file named **Course.cs** within the Entities folder, then copy and paste the following code. In the Course class, we have a Teacher reference navigation property named OnlineTeacher, establishing a one-to-one relationship between the Course and the Teacher.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; }
        public string? CourseName { get; set; }
        public string? Description { get; set; }
        public int? OnlineTeacherId { get; set; }
    }
}
```

```

        public Teacher? OnlineTeacher { get; set; }
    }
}

```

Modifying the Context Class:

Next, modify the EFCoreDbContext class as follows:

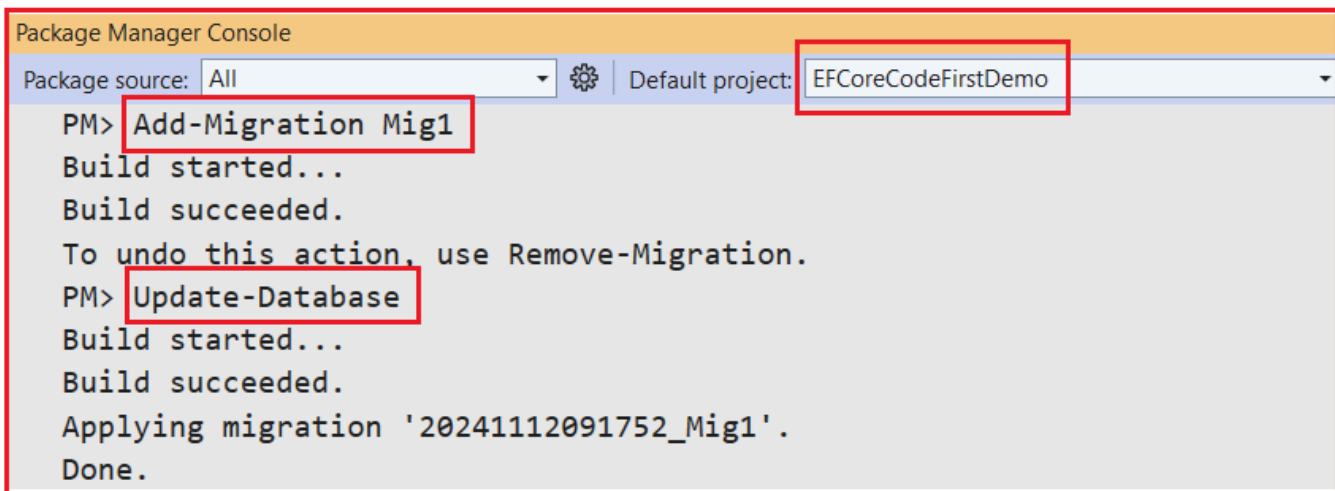
```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Course> Courses { get; set; }
        public DbSet<Teacher> Teachers { get; set; }
    }
}

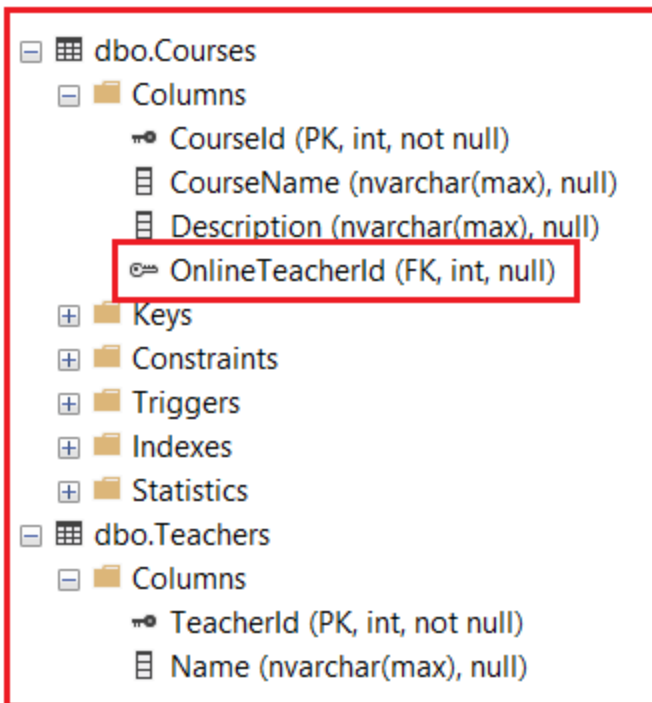
```

Generating Migration and Updating Database

So, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.



Once the above commands are executed successfully, EF Core will create the Courses table with a foreign key (OnlineTeacherId) that references the Teachers table, as shown in the image below.



Creating Multiple Relationships Between Teachers and Course Entities:

Suppose we require to add another relationship: we need an offline teacher for each course in addition to an online teacher. This leads to multiple relationships between the Teacher and Course entities.

First, Let us modify the **Teacher** class to include the OfflineCourses collection Navigation Property. We have added an OfflineCourses collection navigation property, which creates another one-to-many relationship between the Teacher and the Course.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; }
        public string? Name { get; set; }
        public ICollection<Course>? OnlineCourses { get; set; }
        public ICollection<Course>? OfflineCourses { get; set; }
    }
}
```

Next, modify the Course Entity as follows. We have added the OfflineTeacherId and OfflineTeacher Reference Navigational properties.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; }
        public string? CourseName { get; set; }
        public string? Description { get; set; }
        public int? OnlineTeacherId { get; set; }
    }
}
```

```

    public Teacher? OnlineTeacher { get; set; }
    public int? OfflineTeacherId { get; set; }
    public Teacher? OfflineTeacher { get; set; }
}
}

```

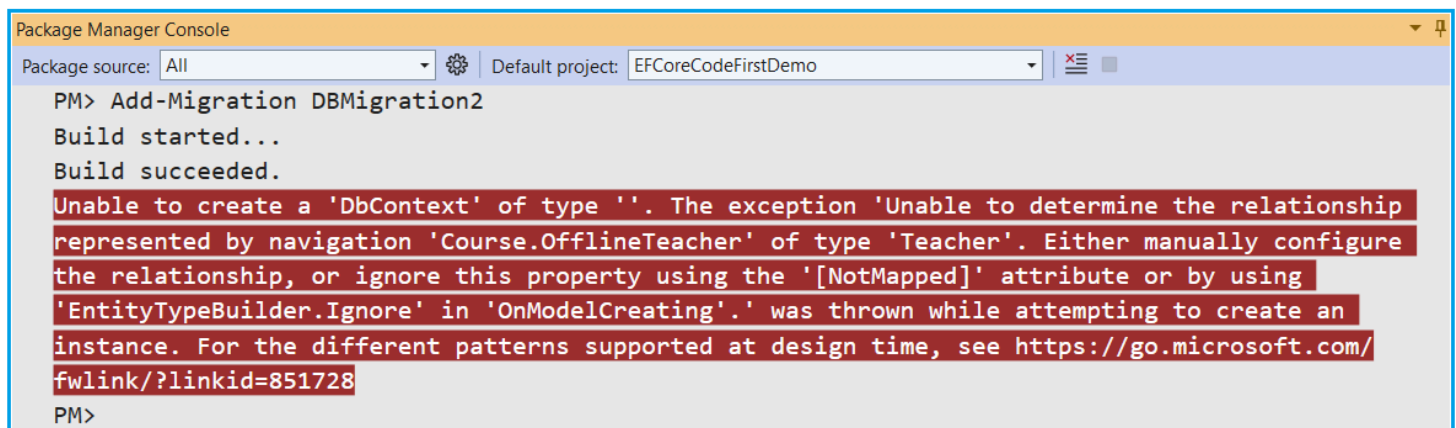
Now, we have two one-to-many relationships between the Course and Teacher entities. A Course can be taught by an online or offline teacher, and a teacher can teach multiple online or offline courses.

Generating the Migration:

EF Core automatically identifies relationships between two entities by default if there is only one pair of navigation properties. However, when multiple relationships exist between the same two entities:

- EF Core cannot determine how to pair the navigation properties.
- This ambiguity results in errors during migration generation.

With the above changes in place, open the Package Manager Console and Execute the following **Add-Migration** command. Now, you should get the following error. It clearly says that it is unable to determine the relationship.



The screenshot shows the Package Manager Console window with the following content:

```

Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> Add-Migration DBMigration2
Build started...
Build succeeded.
Unable to create a 'DbContext' of type ''. The exception 'Unable to determine the relationship represented by navigation 'Course.OfflineTeacher' of type 'Teacher'. Either manually configure the relationship, or ignore this property using the '[NotMapped]' attribute or by using 'EntityTypeBuilder.Ignore' in 'OnModelCreating'.' was thrown while attempting to create an instance. For the different patterns supported at design time, see https://go.microsoft.com/fwlink/?linkid=851728
PM>

```

Why the above Error?

When multiple relationships exist between two entities, Entity Framework Core encounters ambiguities in determining how to map the navigation properties. The following is the Problem Scenario:

- The Teacher has two navigation properties: OnlineCourses and OfflineCourses.
- The Course has two navigation properties: OnlineTeacher and OfflineTeacher.

EF Core cannot automatically determine which Course navigation property pairs with which Teacher navigation property. Hence, EF Core gives the above error.

How Can We Overcome This Problem in EF Core?

To overcome this problem, we need to use the **InverseProperty** Attribute in EF Core. If you go to the definition of InverseProperty class, you will see the following. The **InverseProperty** class has one constructor, which takes a string parameter, and a string read-only property, which will be initialized through the constructor.

```

namespace System.ComponentModel.DataAnnotations.Schema
{
    ... public class InversePropertyAttribute : Attribute
    {
        ... public InversePropertyAttribute(string property);
        ... public string Property { get; }
    }
}

```

Using InverseProperty Attribute in Entity Framework Core

Let us modify the **Teacher** Entity class as follows to use the InverseProperty Attribute. Here, we have decorated the **InverseProperty** Attribute with both **OnlineTeacher** and **OfflineTeacher** properties and specified the Teacher Entity Collection Navigation properties. With this, **OnlineTeacher** will have a relationship with the **OnlineCourses** property, and **OfflineTeacher** will have a relationship with the **OfflineCourses** property.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; }
        public string? CourseName { get; set; }
        public string? Description { get; set; }
        public int? OnlineTeacherId { get; set; }
        [InverseProperty("OnlineCourses")]
        public Teacher? OnlineTeacher { get; set; }
        public int? OfflineTeacherId { get; set; }
        [InverseProperty("OfflineCourses")]
        public Teacher? OfflineTeacher { get; set; }
    }
}

```

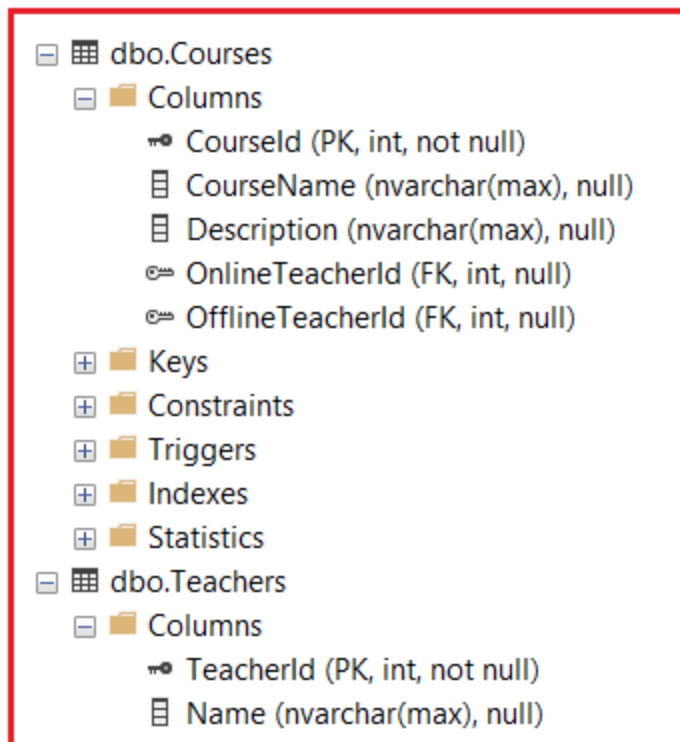
Here:

- **[InverseProperty("OnlineCourses")]** tells EF Core that the OnlineCourses collection is related to the OnlineTeacher property.
- **[InverseProperty("OfflineCourses")]** tells EF Core that the OfflineCourses collection is related to the OfflineTeacher property in the Course entity.

With the above changes, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows. This time, they should be executed as expected.

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v] [x] [ ]
PM> add-migration DBMigration2
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230809115945_DBMigration2'.
Done.
```

Now, verify the database, and you should see the following: In this case, Entity Framework Core created foreign keys OnlineTeacherId and OfflineTeacherId as expected.



Can we apply the InverseProperty Attribute in the Principal Entity?

Yes, we can apply the InverseProperty Attribute either to the Principal or Dependent entity navigation properties. In the previous example, we applied the InverseProperty Attribute to the Course entity reference navigation properties. Now, let us proceed and apply the InverseProperty Attribute to the Teacher entity.

Modify the Course Entity:

First, modify the Course entity as follows. Here, we remove the InverseProperty Attribute from the reference navigation properties.

```
namespace EFCoreCodeFirstDemo.Entities
{
```

```

public class Course
{
    public int CourseId { get; set; }
    public string? CourseName { get; set; }
    public string? Description { get; set; }
    public int? OnlineTeacherId { get; set; }
    public Teacher? OnlineTeacher { get; set; }
    public int? OfflineTeacherId { get; set; }
    public Teacher? OfflineTeacher { get; set; }
}
}

```

Modify the Teacher Entity:

Next, modify the Teacher entity as follows. Here, we are adding the InverseProperty Attribute into the collection navigation properties.

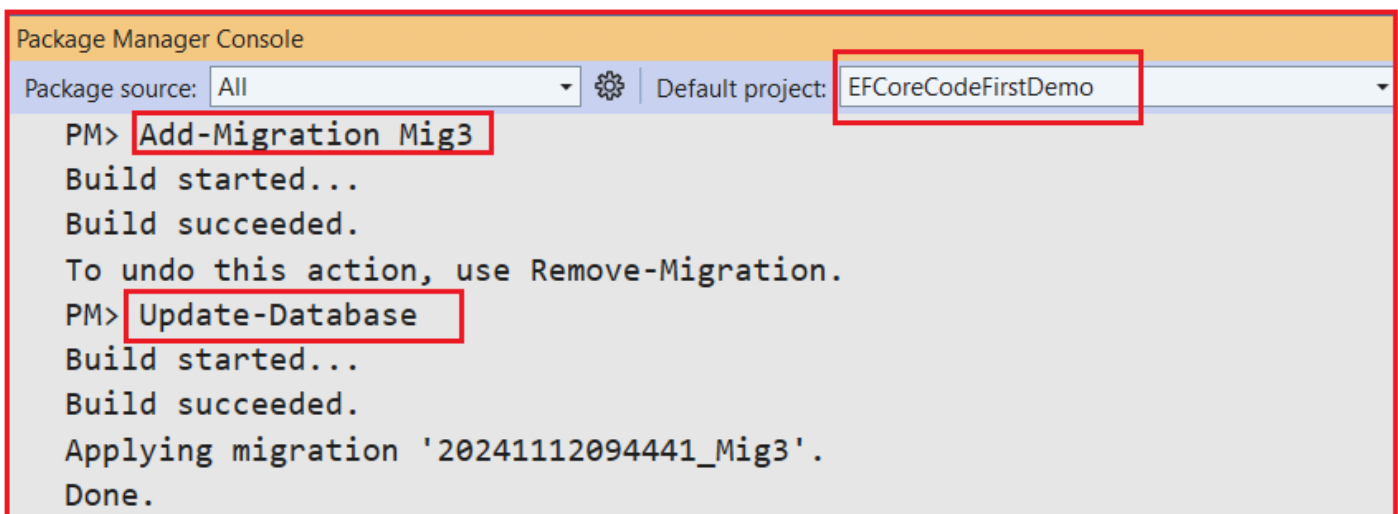
```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; }
        public string? Name { get; set; }
        [InverseProperty("OnlineTeacher")]
        public ICollection<Course>? OnlineCourses { get; set; }
        [InverseProperty("OfflineTeacher")]
        public ICollection<Course>? OfflineCourses { get; set; }
    }
}

```

Generating and Applying Migration:

With the above changes, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.



The screenshot shows the Package Manager Console window. The 'Package source' is set to 'All' and the 'Default project' is 'EFCoreCodeFirstDemo'. The console output shows the following commands and their results:

```

PM> Add-Migration Mig3
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20241112094441_Mig3'.
Done.

```

Now, verify the database, and you should see the database tables with proper foreign keys as expected.

Key Points:

- The InverseProperty attribute can be applied to either side of the relationship: Principal or Dependent Entity.
- It accepts a string parameter that specifies the name of the corresponding navigation property on the other entity.
- Ensure the property names provided to InverseProperty are accurate to avoid runtime errors.

So, we need to use the InverseProperty attribute whenever we encounter multiple relationships between the same entities that EF Core cannot resolve independently. It provides a straightforward way to define relationships explicitly, ensuring that our entity mappings are clear, unambiguous, and function as intended within our application.

In the next article, I will discuss [NotMapped Attribute in Entity Framework Core](#) with Examples. Here, in this article, I try to explain the InverseProperty Data Annotation Attribute in Entity Framework Core with Examples. I hope you enjoyed this InverseProperty Attribute in EF Core with Examples article.